

Project of Core Java

Name = Lokesh Jakhar

Course-name = Core Java Video Training

Course start date = 12/06/2025

Course completion date = 12/07/2025

Project topic = E– Library Management System,
Hotel Booking System

Subject = Core Java

Trainer name = Sharik Ahmed

Mode of Class = Online

Class timing = Video Training

Mini Project

E – Library Management System:

Creating an E-Library Management System where the user has the following options:

- Add Book Information
- Display Book Information
- To list all books of a given author
- To list the count of books in the library

Functionalities Required:

- If the user tries to add a book then the user must have to provide the below specific information about the book:
 - > Enter Book Name:
 - > Enter Author Name:
 - > Enter Pages:
 - > Enter Price:
- When the user tries to display all books of a particular author then the user must have to enter the name of the author:
 - > Enter the Author's Name:
- The E-Library Management System must be also capable of counting all the books available the library.

Ans.

```
import java.util.ArrayList;
import java.util.Scanner;
class Book {
    String bookName;
    String authorName;
    int pages;
    double price;
    public Book(String bookName, String authorName, int
pages, double price) {
        this.bookName = bookName;
        this.authorName = authorName;
        this.pages = pages;
        this.price = price;
    }
    public void displayInfo() {
        System.out.println("Book Name: " + bookName);
        System.out.println("Author Name: " + authorName);
        System.out.println("Pages: " + pages);
        System.out.println("Price: " + price);
        System.out.println("-----");
    }
}
public class project1 {
    static ArrayList<Book> library = new ArrayList<>();
    static Scanner sc = new Scanner(System.in);
    public static void main(String[] args) {
```

```
int choice;
do {
    System.out.println("\n===== E-Library
Management System =====");
    System.out.println("1. Add Book Information");
    System.out.println("2. Display All Book
Information");
    System.out.println("3. List All Books by a
Given Author");
    System.out.println("4. Count Total Number of
Books");
    System.out.println("5. Exit");
    System.out.print("Enter your choice (1-5): ");
    choice = sc.nextInt();
    sc.nextLine();
    switch (choice) {
        case 1:
            addBook();
            break;
        case 2:
            displayAllBooks();
            break;
        case 3:
            listBooksByAuthor();
            break;
        case 4:
            countBooks();
            break;
        case 5:
```

```

        System.out.println("Exiting... Thank
you for using the E-Library System!");
        break;
        default:
            System.out.println("Invalid choice!
Please try again.");
    }
} while (choice != 5);
}

public static void addBook() {
    System.out.print("Enter Book Name: ");
    String bookName = sc.nextLine();
    System.out.print("Enter Author Name: ");
    String authorName = sc.nextLine();
    System.out.print("Enter Pages: ");
    int pages = sc.nextInt();
    System.out.print("Enter Price: ");
    double price = sc.nextDouble();
    sc.nextLine();
    Book newBook = new Book(bookName, authorName,
pages, price);
    library.add(newBook);
    System.out.println("Book added successfully!");
}

public static void displayAllBooks() {
    if (library.isEmpty()) {
        System.out.println("No books in the library.");
        return;
    }
}

```

```

        System.out.println("\n--- Book Information ---");
        for (Book b : library) {
            b.displayInfo();
        }
    }

    public static void listBooksByAuthor() {
        System.out.print("Enter the Author's Name: ");
        String author = sc.nextLine();
        boolean found = false;
        for (Book b : library) {
            if (b.authorName.equalsIgnoreCase(author)) {
                b.displayInfo();
                found = true;
            }
        }
        if (!found) {
            System.out.println("No books found for the
author: " + author);
        }
    }

    public static void countBooks() {
        System.out.println("Total number of books in the
library: " + library.size());
    }
}

```

Major Project

Hotel Booking System

Features:

- Add/View room types and availability
- Book a room for customer
- Customer check-in & check-out
- Billing based on days and room type
- Save customer and booking data in files

Concepts Used:

- Classes & Objects (Room, Customer, Booking)
- File I/O for data storage
- Exception Handling for invalid inputs
- LocalDate for date calculations

Ans.

```
import java.io.*;
import java.time.LocalDate;
import java.time.temporal.ChronoUnit;
import java.util.*;
class Room {
    String type;
    double pricePerDay;
    boolean isAvailable;
    public Room(String type, double pricePerDay) {
```

```

        this.type = type;
        this.pricePerDay = pricePerDay;
        this.isAvailable = true;
    }
    public void displayInfo() {
        System.out.println("Room Type: " + type);
        System.out.println("Price per Day: " +
pricePerDay);
        System.out.println("Availability: " +
(isAvailable ? "Available" : "Booked"));
        System.out.println("-----
");
    }
}
class Customer {
    String name;
    String contact;
    LocalDate checkInDate;
    LocalDate checkOutDate;
    public Customer(String name, String contact, LocalDate
checkInDate, LocalDate checkOutDate) {
        this.name = name;
        this.contact = contact;
        this.checkInDate = checkInDate;
        this.checkOutDate = checkOutDate;
    }
}
class Booking {
    Customer customer;

```



```

    Room room;

    public Booking(Customer customer, Room room) {
        this.customer = customer;
        this.room = room;
    }

    public long getStayDuration() {
        return
ChronoUnit.DAYS.between(customer.checkInDate,
customer.checkOutDate);
    }

    public double calculateBill() {
        return getStayDuration() * room.pricePerDay;
    }

    public void printBill() {
        System.out.println("==== BILL =====");
        System.out.println("Customer: " + customer.name);
        System.out.println("Room Type: " + room.type);
        System.out.println("Stay Duration: " +
getStayDuration() + " day(s)");
        System.out.println("Total Amount: " +
calculateBill());
        System.out.println("=====");
    }

    public String toFileString() {
        return customer.name + ";" + customer.contact +
";" + room.type + ";" + customer.checkInDate + ";" +
customer.checkOutDate + ";" + calculateBill();
    }
}

```

```

public class project2 {
    static Scanner sc = new Scanner(System.in);
    static ArrayList<Room> rooms = new ArrayList<>();
    static ArrayList<Booking> bookings = new ArrayList<>();
    static final String FILE_NAME = "bookings.txt";
    public static void main(String[] args) {
        loadSampleRooms();
        int choice;
        do {
            System.out.println("\n===== Hotel Booking
System =====");
            System.out.println("1. View Room Types &
Availability");
            System.out.println("2. Book a Room");
            System.out.println("3. Check-out & Generate
Bill");
            System.out.println("4. Save Bookings to File");
            System.out.println("5. Exit");
            System.out.print("Enter your choice (1-5): ");
            try {
                choice = Integer.parseInt(sc.nextLine());
            } catch (Exception e) {
                System.out.println("Invalid input! Please
enter a number.");
                continue;
            }
            switch (choice) {
                case 1 : viewRooms();
                break;

```

```

        case 2 : bookRoom();
        break;
        case 3 : checkOutCustomer();
        break;
        case 4 : saveToFile();
        break;
        case 5 : System.out.println("Exiting...
Thank you!");
        return;
        default : System.out.println("Invalid
choice!");
    }
} while (true);
}

public static void loadSampleRooms() {
    rooms.add(new Room("Single", 1000));
    rooms.add(new Room("Double", 2000));
    rooms.add(new Room("Deluxe", 3000));
}

public static void viewRooms() {
    System.out.println("\n--- Room Types and
Availability ---");
    for (Room room : rooms) {
        room.displayInfo();
    }
}

public static void bookRoom() {
    System.out.print("Enter your name: ");
    String name = sc.nextLine();

```

```

        System.out.print("Enter contact number: ");
        String contact = sc.nextLine();
        System.out.print("Enter check-in date (YYYY-MM-DD): ");
        String checkInStr = sc.nextLine();
        LocalDate checkIn = LocalDate.parse(sc.nextLine());
        System.out.print("Enter check-out date (YYYY-MM-DD): ");
        String checkOutStr = sc.nextLine();
        LocalDate checkOut = LocalDate.parse(sc.nextLine());
        viewRooms();
        System.out.print("Enter room type to book: ");
        String type = sc.nextLine();
        for (Room room : rooms) {
            if (room.type.equalsIgnoreCase(type) &&
room.isAvailable) {
                room.isAvailable = false;
                Customer customer = new Customer(name,
contact, checkIn, checkOut);
                Booking booking = new Booking(customer,
room);
                bookings.add(booking);
                System.out.println("Room booked
successfully!");
                return;
            }
        }
        System.out.println("Sorry, no available rooms of
that type.");
    }

```

```

    public static void checkOutCustomer() {
        System.out.print("Enter your name for checkout: ");
        String name = sc.nextLine();
        for (Booking b : bookings) {
            if (b.customer.name.equalsIgnoreCase(name)) {
                b.printBill();
                b.room.isAvailable = true;
                bookings.remove(b);
                System.out.println("Check-out
successful.");
                return;
            }
        }
        System.out.println("No booking found under this
name.");
    }

    public static void saveToFile() {
        try (PrintWriter writer = new PrintWriter(new
FileWriter(FILE_NAME))) {
            for (Booking b : bookings) {
                writer.println(b.toFileString());
            }
            System.out.println("Bookings saved to file.");
        } catch (IOException e) {
            System.out.println("Error saving bookings to
file.");
        }
    }
}

```